# Lab 45: Prove the template before 35,000 transcripts

Topic 4: Prompt Engineering & Structured Output · Task 4.5: Design efficient batch processing strategies
Runs on Simulator 1: Customer Support Resolution Agent · Run in class
Source: SkillCertPro practice exam Part 1, Q1 (unofficial third-party question)

## The question

Before enabling overnight processing of 35,000 archived support transcripts through the Message Batches API, a pilot of 30 transcripts shows inconsistent disposition decisions and missing rationale fields in some outputs. A product manager proposes running the full archive now and resubmitting bad rows the next day. What should this team do next?

**A.** Submit the full archive as one batch, review the ended results, and resubmit rows whose disposition or fields are wrong.

**B.** Run the prompt improver once on the current template, accept the revised prompt, and start the full overnight batch.

**C.** Create a representative evaluation set, refine the templated prompt against clear criteria, and rerun it before submitting the full batch.

**D.** Add stricter JSON output constraints, keep the current instructions, and use the batch results to discover decision errors.

## Simulate it live

```
# In the browser: open Simulator 1, then Lab 45
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 45

# In a terminal, from simulators/simulator-1-customer-support/
uv run triage.py --pilot
```

## What you should see

The refined template reports it is ready for the overnight batch.

evidence/triage-triage_template_v0.json has a lower first-pass rate than evidence/triage-triage_template.json; only the latter reports ready_for_batch true.

Open `simulators/simulator-1-customer-support/evidence/` and find it yourself.

## The answer

**C.** Evaluation set, refined template, rerun, then batch.

For a large asynchronous workload the goal is to maximise first-pass success before scale. The video frames prompt engineering as an evaluation loop: define success criteria, build tests that mirror real inputs (routine returns, billing disputes, explicit requests for a human, policy edge cases, messy customer language), iterate on the prompt, validate, then ship. Prompt templates separate fixed instructions from variables such as transcript , customer_history and policy_excerpt , and Console evaluations let you compare prompt versions across test cases. Batch requests are processed independently and asynchronously, so a batch is a poor place to discover preventable prompt defects at full volume. Structured outputs help format reliability, but semantic correctness still needs evaluation.

## Why the others fail

**A.** Uses the 35,000-item batch as the test harness. The pilot already shows systematic defects, so most of the rework is preventable; resubmitting failures by custom_id is for residual errors, not known prompt problems.

**B.** One unvalidated pass of the prompt improver, accepted without testing against criteria, gives no evidence that the disposition inconsistency is fixed.

**D.** JSON constraints may fix the missing fields (format), but not the inconsistent decisions (semantics), and errors are still discovered only after the full run.

## Check yourself

- the v1 template fills {{transcript}}, {{transcript_id}} and {{policy_excerpt}}
- first-pass success with v1 is 100% on the ten pilot transcripts
- the learner can state why resubmitting bad rows tomorrow costs more than piloting

## Now change one thing

Copy a file in cases/ to a new CS id, change the customer's message and expected disposition, rerun with --case, then score it with evaluate.py.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.